Business Requirements Document (BRD)

Hospital Management System — SQL Analysis Project

1. Project background

A hospital stores information about its departments, doctors, patients, and appointments. Management needs a simple way to review patient activity, doctor workload, appointment outcomes, and treatment revenue.

This project uses a fictional dataset and SQL queries to answer those business questions.

2. Business objective

Build a SQL-based analysis that helps hospital management:

- Understand patient registrations and locations.
- Review doctor availability and department-level performance.
- Monitor appointment completion, pending, and cancellation counts.
- Analyze treatment revenue and payment methods.
- Identify patient visit patterns and patient feedback.

3. Scope

In scope

- Create and populate a relational database with four tables.
- Analyze patient, doctor, department, and appointment data.
- Calculate operational and revenue KPIs.
- Use joins, aggregate functions, subqueries, CTEs, and window functions.

Out of scope

- Real patient records or protected health information.
- Appointment booking application or user interface.
- Inventory, pharmacy, staff payroll, insurance claim settlement, or clinical diagnosis.
- Predictive analytics and machine learning.

4. Database overview

Table	Key fields	Description
Departments	department_id, department_name, floor_no	Hospital department details
Doctors	doctor_id, department_id, doctor_name, experience_years, consultation_fee	Doctor details and department assignment
Patients	patient_id, patient_name, age, blood_group, city, registration_date	Patient registration details
Appointments	appointment_id, patient_id, doctor_id, appointment_date, disease, treatment_cost, payment_mode, status, rating	Appointment and treatment details

5. Business rules and assumptions

1. Each doctor belongs to a department through `department_id`.
2. Each appointment references one patient and one doctor.
3. Appointment status values in the supplied data are `Completed`, `Pending`, and `Cancelled`.
4. Revenue and completed-visit KPIs include only appointments with `status = 'Completed'`.
5. Pending and cancelled appointments are included when reporting appointment status, but excluded from completed revenue.
6. `rating` can be NULL when feedback has not been recorded.
7. All dataset records are fictional and intended for portfolio demonstration.

6. Key performance indicators (KPIs)

KPI	Definition
Total patients	Count of records in Patients
Total doctors	Count of records in Doctors
Completed appointments	Count where status is Completed
Pending appointments	Count where status is Pending
Cancelled appointments	Count where status is Cancelled
Completed treatment revenue	Sum of treatment_cost for Completed appointments
Appointment completion rate	Completed appointments / all appointments × 100
Average patient rating	Average non-NULL rating for completed appointments

7. Business requirements and SQL questions

A. Data exploration and filtering

BR-01. List the unique cities where patients are registered.

BR-02. Display appointments ordered by appointment date.

BR-03. Identify the five most recently registered patients.

BR-04. Retrieve all completed appointments.

BR-05. Identify appointments with treatment costs greater than ■5,000.

BR-06. Identify patients older than 40 years.

BR-07. Identify doctors with more than 10 years of experience.

BR-08. Retrieve appointments paid through Insurance.

BR-09. Find patients aged between 25 and 40 years.

BR-10. Retrieve appointments that are Pending or Cancelled.

BR-11. Identify appointments that do not have a rating.

B. Summary and aggregate analysis

BR-12. Calculate the total number of registered patients.

BR-13. Calculate the average doctor consultation fee.

BR-14. Find the highest treatment cost recorded.

BR-15. Calculate completed treatment revenue.

BR-16. Calculate the average treatment cost for completed appointments.

BR-17. Count completed appointments.

BR-18. Calculate completed treatment revenue from Insurance payments.

C. Department and doctor performance

BR-19. Count registered patients city-wise.

BR-20. Calculate average consultation fee by department.

BR-21. Calculate completed treatment revenue by doctor.

BR-22. Count completed appointments handled by each doctor.

BR-23. Identify departments with more than one doctor.

BR-24. Calculate completed treatment revenue by department.

BR-25. Calculate the average patient rating for each doctor.

D. Business analysis

BR-26. Identify the city with the highest number of registered patients.

BR-27. Identify the doctor with the highest completed treatment revenue.

BR-28. Identify the patient with the highest number of completed visits.

BR-29. Identify the most commonly treated disease among completed appointments.

BR-30. Identify patients who have consulted more than one distinct doctor.

BR-31. Identify doctors who have not received a rating below 4, considering doctors with at least one rated completed appointment.

BR-32. Identify departments whose average completed treatment cost is above the hospital-wide average.

E. KPI, CTE, and window-function analysis

BR-33. KPI dashboard summary. Show total patients, total doctors, completed appointments, pending appointments, cancelled appointments, and completed treatment revenue.

BR-34. Appointment completion rate. Calculate the percentage of all appointments that are completed.

BR-35. CTE — Patient visit and spend analysis. Find patients with at least two completed visits and show their total completed treatment spend.

BR-36. CTE — Top doctors by revenue. Find the top three doctors by completed treatment revenue.

BR-37. Window function — Department-wise doctor ranking. Rank doctors by completed treatment revenue within each department using `DENSE_RANK()`.

BR-38. Window function — Previous visit comparison. For each patient, show the treatment cost of their previous completed appointment using `LAG()`.

8. Expected outcome

The project should provide a set of reusable SQL queries that answer the requirements above and demonstrate how relational data can be converted into useful operational and management insights.

9. Tools

- MySQL 8.0+

- MySQL Workbench
- GitHub for version control and project presentation